מטלה 3 - רשתות תקשורת

מגישים: אריאל יעקובי ויואב נחמני
ת"ז: 318727187 + 213096084

המטלה הוגשה כקובץ ZIP המכיל:
א)

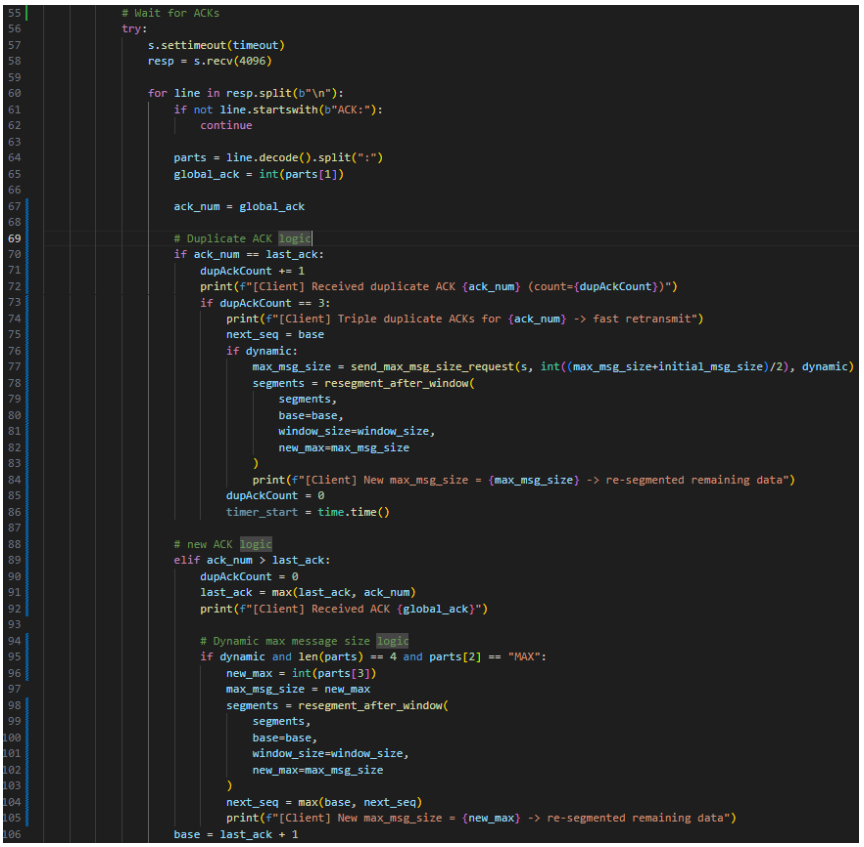
חלק 1:

ראשית, אפרט על **מבנה הקבצים בפרויקט**.

client.py בנוי מהפונקציות הבאות:

handle_client: זו פונקציית הליבה של הלקוח ומנהלת את כל תהליך התקשורת עם השרת. הפונקציה קוראת את פרטי הקונפיגורציה, טוענת את ההודעה מקובץ כ־bytes תוך נרמול תווי ירידת שורה, ויוצרת חיבור TCP לשרת עם timeout מוגדר. לאחר מכן מתבצע Handshake ידני (SIN / SIN-ACK / ACK), בקשה לגודל ההודעה המקסימלי, חלוקת ההודעה למקטעים ושליחה באמצעות Sliding Window. הפונקציה מטפלת ב־ACK מצטבר, duplicate ACK, ב־timeouts ובהעברות חוזרות. במצב Dynamic Message Size היא מעדכנת את גודל המקטעים שאחרי החלון הנוכחי בזמן ריצה לפי ערכי MAX שמתקבלים מהשרת מבלי לשנות את החלון הנוכחי. במקרה של שלוש ACK משוכפלים הלקוח יבקש להוריד את גודל המקטע המקסימלי לממוצע בין הגודל הנוכחי לגודל ההתחלתי וב- timeout הלקוח יבקש מהשרת להוריד את גודל המקטע המקסימלי חזרה לגודל ההתחלתי. בסיום כל הנתונים נשלחת הודעת FIN והחיבור נסגר.

```python
# ------------------- Client Logic ------------------- #

def handle_client(host, port, config):
    message_path = config["message"]
    initial_msg_size = config["maximum_message_size"]
    window_size = config["window_size"]
    timeout = config["timeout"]          # seconds
    dynamic = config["dynamic_message_size"]

    with open(message_path, "rb") as f:
        content = f.read()
        message_bytes = content.replace(b"\r\n", b" ").replace(b"\n", b" ")


    with socket.create_connection((host, port)) as s:
        s.settimeout(timeout)

        # ---------- Handshake ----------
        Hand_shake(s)

        # ---------- Get max message size ----------
        max_msg_size = send_max_msg_size_request(s, initial_msg_size, dynamic)

        print(f"[Client] Initial max message size = {max_msg_size} bytes")

        # ---------- Segmentation ----------
        segments = segment_bytes(message_bytes, max_msg_size)

        base = 0
        next_seq = 0
        last_ack = -1
        timer_start = None

        dupAckCount = 0
        while base < len(segments):

            # Send window
            while next_seq < len(segments) and next_seq - base < window_size:
                payload = segments[next_seq]
                header = f"M{next_seq}:".encode()

                s.sendall(header + payload + b"\n")
                print(f"[Client] Sent segment {next_seq}")

                if base == next_seq:
                    timer_start = time.time()
                next_seq += 1

```



```python
        # Wait for ACKs
        try:
            s.settimeout(timeout)
            resp = s.recv(4096)

            for line in resp.split(b"\n"):
                if not line.startswith(b"ACK:"):
                    continue

                parts = line.decode().split(":")
                global_ack = int(parts[1])

                ack_num = global_ack

                # Duplicate ACK logic
                if ack_num == last_ack:
                    dupAckCount += 1
                    print(f"[Client] Received duplicate ACK {ack_num} (count={dupAckCount})")
                    if dupAckCount == 3:
                        print(f"[Client] Triple duplicate ACKs for {ack_num} -> fast retransmit")
                        next_seq = base
                        if dynamic:
                            max_msg_size = send_max_msg_size_request(s, int((max_msg_size+initial_msg_size)/2), dynamic)
                            segments = resegment_after_window(
                                segments,
                                base=base,
                                window_size=window_size,
                                new_max=max_msg_size
                            )
                            print(f"[Client] New max_msg_size = {max_msg_size} -> re-segmented remaining data")
                        dupAckCount = 0
                        timer_start = time.time()

                # new ACK logic
                elif ack_num > last_ack:
                    dupAckCount = 0
                    last_ack = max(last_ack, ack_num)
                    print(f"[Client] Received ACK {global_ack}")

                    # Dynamic max message size logic
                    if dynamic and len(parts) == 4 and parts[2] == "MAX":
                        new_max = int(parts[3])
                        max_msg_size = new_max
                        segments = resegment_after_window(
                            segments,
                            base=base,
                            window_size=window_size,
                            new_max=max_msg_size
                        )
                        next_seq = max(base, next_seq)
                        print(f"[Client] New max_msg_size = {new_max} -> re-segmented remaining data")
                    base = last_ack + 1
```

```python
                    if base == next_seq:
                        timer_start = None
                    else:
                        timer_start = time.time()

        except socket.timeout:
            print("[Client] Timeout -> retransmitting window")
            if dynamic:
                max_msg_size = send_max_msg_size_request(s, initial_msg_size, dynamic)
                segments = resegment_after_window(
                    segments,
                    base=base,
                    window_size=window_size,
                    new_max=max_msg_size
                )
                print(f"[Client] New max_msg_size = {max_msg_size} -> re-segmented remaining data")
            next_seq = base
            dupAckCount = 0
            timer_start = time.time()

    # ---------- End of transmission ----------
    s.sendall(b"FIN\n")
    print("[Client] Transmission complete")
```

Hand_shake: פונקציית עזר שמבצעת לחיצת יד משולשת עם השרת.

send_max_msg_size_request: פונקציית עזר ששולחת בקשה לשרת עם גודל הבקשה המקסימלי המבוקש

```python
# ------------------- Server requests ------------------- #


def Hand_shake(s):
    """Performs a three-way handshake with the server."""
    while True:
        s.sendall(b"SIN\n")
        try:
            if b"SIN/ACK" in s.recv(1024):
                s.sendall(b"ACK\n")
                break
        except socket.timeout:
            s.sendall(b"SIN\n")

def send_max_msg_size_request(s, max_msg_size, dynamic)->int:
    """Sends a GetMaxMsgSize request to the server and returns the max message size."""
    while True:
            s.sendall(f"GetMaxMsgSize:{max_msg_size},{dynamic}\n".encode())
            try:
                resp = s.recv(1024)
                if resp.startswith(b"MaxMsgSize:"):
                    max_msg_size = int(resp.split(b":")[1])
                    return max_msg_size
            except socket.timeout:
                s.sendall(f"GetMaxMsgSize:{max_msg_size},{dynamic}\n".encode())

```

segment_bytes: פונקציית עזר שמחלקת מערך bytes לרשימת מקטעים בגודל מקסימלי נתון.
פונקציה זו משמשת לביצוע segmentation מדויק ברמת בתים.

```python
# ------------------- Utilities ------------------- #

def segment_bytes(data: bytes, max_size: int):
    return [data[i:i+max_size] for i in range(0, len(data), max_size)]
```

resegment_after_window: פונקציית עזר שמחלקת את רשימת הבתים לקטעים בגודל המקסימלי החדש רק אחרי החלון הנוכחי

```python
    def resegment_after_window(
        segments: list[bytes],
        *,
        base: int,
        window_size: int,
        new_max: int
    ) -> list[bytes]:
        """
        Re-segments ONLY the data after the current window using new_max.
        Everything before and inside the window remains unchanged.
        """

        window_end = min(base + window_size, len(segments))

        # חלקים שלא נוגעים בהם
        before = segments[:window_end]

        # כל הדאטה שאחרי החלון
        after_data = b"".join(segments[window_end:])

        # פירוק מחדש של מה שאחרי החלון
        after_segments = [
            after_data[i:i + new_max]
            for i in range(0, len(after_data), new_max)
        ]

        return before + after_segments
```

readFile: פונקציה זו קוראת קובץ קונפיגורציה בפורמט טקסטואלי וממירה אותו למילון Python. היא מטפלת בנרמול מפתחות, המרת ערכים מספריים וקריאת ערכי boolean, ומאפשרת להגדיר את פרמטרי הריצה ללא שינוי בקוד.

```python
def readFile(f):
    config = {}
    for line in f:
        line = line.strip()
        if not line or line.startswith("#"):
            continue

        key, value = line.split(":", 1)
        key = key.strip().lower().replace(" ", "_")
        value = value.strip()

        if key == "dynamic_message_size":
            config[key] = value.lower() == "true"
        elif key == "message":
            config[key] = value.strip('"')
        elif value.isdigit():
            config[key] = int(value)

    return config
```

interactive_config: פונקציה זו מאפשרת הרצה אינטראקטיבית של הלקוח ללא קובץ קונפיגורציה. המשתמש מזין את פרטי ההודעה והפרמטרים דרך הקונסול, והפונקציה מפעילה את `handle_client` עם הערכים שהוזנו.

```python
def interactive_config(host, port):
    config = {
        "message": input("Message file path: ").strip(),
        "maximum_msg_size": int(input("Maximum message size (bytes): ").strip()),
        "window_size": int(input("Window size: ").strip()),
        "timeout": int(input("Timeout (seconds): ").strip()),
        "dynamic_message_size": input("Dynamic message size? (y/n): ").lower() == "y"
    }
    handle_client(host, port, config)
```

main: הפונקציה הראשית אחראית על ניתוח פרמטרים משורת הפקודה. היא קובעת האם הלקוח יורץ באמצעות קובץ קונפיגורציה או במצב אינטראקטיבי, ולאחר מכן מפעילה את פונקציית `handle_client` לביצוע התקשורת.

```python
# ------------------- Main ------------------- #

def main():
    ap = argparse.ArgumentParser(description="Reliable client over TCP")
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=5555)
    ap.add_argument("--config", type=str)

    args = ap.parse_args()

    if args.config:
        with open(args.config, "r") as f:
            config = readFile(f)
    else:
        return interactive_config(args.host, args.port)

    handle_client(args.host, args.port, config)


if __name__ == "__main__":
    main()
```

server.py בנוי מהפונקציות הבאות:

handle_client():

זו פונקציית הליבה של השרת ומטפלת בלקוח יחיד. הפונקציה מאתחלת את פרטי החיבור ומתחילה להאזין לנתונים מהלקוח. השרת מבצע Handshake ידני על ידי קבלת SIN ושליחת SIN/ACK, משיב לבקשת GetMaxMsgSize, ומקבל מקטעי נתונים עם מספרי רצף. עבור כל מקטע שמתקבל, השרת שולח ACK מצטבר עבור המקטע האחרון שהתקבל ברצף, ובמצב דינמי מעדכן את גודל המקטעים ומעביר את הערך החדש ללקוח דרך שדה MAX. לאחר קבלת FIN והרכבת כל המקטעים, השרת מחבר את ההודעה המלאה ומדפיס אותה. מכיוון שאנו עובדים על loacal host אין איבוד חבילות לכן מימשנו איבוד חבילות על ידי התעלמות מחבילות לפי הסתברות מסויימת.

```python
def handle_client(conn: socket.socket, addr):
    print(f"[server] connected to {addr}")

    max_msg_size = 1  # Default max message size
    dynamic = False  # Default dynamic behavior

    segments = {}
    highest_seq = -1
    received_fin = False

    with conn:
        buffer = b""

        while True:
            data = conn.recv(4096)
            if not data:
                break

            buffer += data

            while b"\n" in buffer:
                line, _, buffer = buffer.partition(b"\n")

                if not line:
                    continue

                if random.random() < lossProbability:
                    print("[server] Simulating packet loss")
                    print("[server] Dropped line:", line)
                    continue
                # ---------- Handshake ----------
                if line == b"SIN":
                    conn.sendall(b"SIN/ACK\n")

                elif line == b"ACK":
                    continue

                # ---------- Max Message Size ----------
                elif line.startswith(b"GetMaxMsgSize"):
                    rest = line.split(b":", 1)[1]
                    req_b, dyn_b = rest.split(b",", 1)
                    max_msg_size = int(req_b)
                    dynamic = dyn_b == b"True"
                    resp = f"MaxMsgSize:{max_msg_size}\n".encode()
                    conn.sendall(resp)
                    print(f"[server] sent MaxMsgSize {max_msg_size}")

                # ---------- FIN ----------
                elif line == b"FIN":
                    received_fin = True
                    print("[server] FIN received")
```

```
            # ---------- Data Segment ----------
            elif line.startswith(b"M"):
                try:
                    header, payload = line[1:].split(b":", 1)
                    seq = int(header)

                    segments[seq] = payload
                    print(f"[server] received segment {seq} ({len(payload)} bytes)")

                    while highest_seq + 1 in segments:
                        highest_seq += 1

                    # Dynamic max message size
                    ack = f"ACK:{highest_seq}"
                    if dynamic:
                        max_msg_size = max_msg_size+1
                        ack += f":MAX:{max_msg_size}"

                    if not received_fin:
                        conn.sendall((ack + "\n").encode())

                except ValueError:
                    print(f"[server] Error parsing line: {line}")
                except OSError:
                    print("[server] client closed connection, stopping ACKs")
                    return


            # ---------- Completion ----------
            if received_fin and highest_seq + 1 == len(segments):
                full_message = b"".join(segments[i] for i in range(len(segments)))
                print("\n[server] COMPLETE MESSAGE RECEIVED:")
                try:
                    print(full_message.decode("utf-8"))
                except UnicodeDecodeError:
                    print(full_message)
                print("[server] end of message\n")
                return
```

serve():

פונקציה זו אחראית על פתיחת הסוקט של השרת והאזנה לחיבורים נכנסים. עבור כל חיבור חדש נוצר thread נפרד, אשר מפעיל את הפונקציה handle_client ומאפשר טיפול במספר לקוחות במקביל.

```
def serve(host, port, config):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((host, port))
        s.listen(5)

        print(f"[server] listening on {host}:{port}")

        while True:
            conn, addr = s.accept()
            threading.Thread(
                target=handle_client,
                args=(conn, addr, config),
                daemon=True
            ).start()
```

main():

הפונקציה הראשית מנתחת את פרמטרי שורת הפקודה, טוענת את קובץ הקונפיגורציה או קוראת ערכים מהמשתמש במצב אינטראקטיבי, ומפעילה את השרת באמצעות הפונקציה serve.

```python
def main():
    ap = argparse.ArgumentParser(description="Reliable TCP Server")
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=5555)
    ap.add_argument("--config", type=str)

    args = ap.parse_args()

    if args.config:
        with open(args.config, "r") as f:
            config = read_config(f)
    else:
        config = {
            "maximum_message_size": int(input("Max message size (bytes): ").strip()),
            "dynamic_message_size": input("Dynamic message size? (y/n): ").lower() == "y"
        }
    print("SERVER CONFIG:", config)

    serve(args.host, args.port, config)


if __name__ == "__main__":
    main()
```

config.txt:

קובץ זה מגדיר את פרמטרי הריצה של הלקוח. הקובץ כולל את נתיב קובץ ההודעה לשליחה, גודל מקסימלי רצוי להודעה, גודל חלון ה־Sliding Window, ערך ה־timeout בשניות, והאם הלקוח פועל במצב Dynamic Message Size. השימוש בקובץ מאפשר לשנות תרחישי הרצה שונים מבלי לשנות את קוד הלקוח.

```
message: message.txt
maximum message size: 20
window_size: 4
timeout: 3
dynamic message size: false
```

message.txt:

קובץ זה מכיל את תוכן ההודעה שנשלחת מהלקוח לשרת. ההודעה משמשת כקלט לפרוטוקול, מחולקת למקטעים בצד הלקוח ומורכבת מחדש בצד השרת, ובכך מאפשרת לוודא שהעברת הנתונים מתבצעת בצורה אמינה ובסדר הנכון.

message.txt

```
Hello,
This is a test message for Assignment 3.
We are testing reliable transport with sliding window,
ACKs, retransmissions and dynamic message size.
```

חלק 2:

כעת נעבור להרצת מצבים שונים. נתחיל ב:

Dynamic Message Size + Sliding Window + Handshake

צד השרת:

```
[server] connected to ('127.0.0.1', 59127)
[server] sent MaxMsgSize 20
[server] Simulating packet loss
[server] Dropped line: b'M0:Hello, This is a tes'
[server] received segment 1 (20 bytes)
[server] received segment 2 (20 bytes)
[server] received segment 3 (20 bytes)
[server] sent MaxMsgSize 20
[server] received segment 0 (20 bytes)
[server] Simulating packet loss
[server] Dropped line: b'M1:t message for Assign'
[server] received segment 2 (20 bytes)
[server] received segment 3 (20 bytes)
[server] received segment 4 (21 bytes)
[server] received segment 5 (21 bytes)
[server] received segment 6 (21 bytes)
[server] received segment 7 (10 bytes)
[server] FIN received

[server] COMPLETE MESSAGE RECEIVED:
Hello, This is a test message for Assignment 3. We are testing reliable transport with sliding window, ACKs, retransmissions and dynamic message size.
[server] end of message
```

השרת עולה עם והלקוח מבקש ממנו maximum_message_size: 20
הלקוח מבקש גם dynamic_message_size: True
כפי שצוין בחלק 1.
נוצר חיבור מהלקוח, השרת שולח MaxMsgsize 20.
השרת "מאבד" את מקטע 0 אבל מקבל את חבילות 1-3, לאחר שמקטע 0 מתקבל השרת "מאבד" את מקטע 1 לאחר שמקטע 1 מתקבל שאר המקטעים מתקבלים כרגיל (מקטעים 1-7) בכל פעם שחבילה חדשה מתקבלת השרת מעלה את גודל המקטע המקסימלי ב-1.

צד הלקוח:

```
C:\dev\assignment3\Assignment_3>py client.py --config C:\dev\assignment3\Assignment_3\config.txt
[Client] Initial max message size = 20 bytes
[Client] Sent segment 0
[Client] Sent segment 1
[Client] Sent segment 2
[Client] Sent segment 3
[Client] Received duplicate ACK -1 (count=1)
[Client] Received duplicate ACK -1 (count=2)
[Client] Received duplicate ACK -1 (count=3)
[Client] Triple duplicate ACKs for -1 -> fast retransmit
[Client] New max_msg_size = 20 -> re-segmented remaining data
[Client] Sent segment 0
[Client] Sent segment 1
[Client] Sent segment 2
[Client] Sent segment 3
[Client] Received ACK 3
[Client] New max_msg_size = 21 -> re-segmented remaining data
[Client] Sent segment 4
[Client] Sent segment 5
[Client] Sent segment 6
[Client] Sent segment 7
[Client] Received duplicate ACK 3 (count=1)
[Client] Received duplicate ACK 3 (count=2)
[Client] Received ACK 4
[Client] New max_msg_size = 24 -> re-segmented remaining data
[Client] Received ACK 5
[Client] New max_msg_size = 25 -> re-segmented remaining data
[Client] Received ACK 6
[Client] New max_msg_size = 26 -> re-segmented remaining data
[Client] Received ACK 7
[Client] New max_msg_size = 27 -> re-segmented remaining data
[Client] FIN acknowledged by server
[Client] Connection closed
[Client] Transmission complete
```

הלקוח מקבל Initial max message size = 20 bytes ולאחר מכן שולח מקטעים 0-3. החבילה הראשונה נופלת ולכן מתקבל ACK משולש, הלקוח שולח שוב את החלון. לאחר מכן מתקבל ACK3 והשרת עדכן max message size = 21, עבור כל ACK חדש שמתקבל max message size גדל ב- 1. במידה והיה timeout אז max message size היה מאותחל חזרה ל-20
לבסוף מודיע Transmission complete.
ניתן לראות שהלקוח משנה מקטעים בזמן ריצה, ומגיב לACK עם MAX.

בוצעה הקלטה עבור המקרה הנ"ל(קובץ pcap_dynamic_mode.pcap בזיפ):
נבצע סינון לפי tcp.port == 5555:

Info	ength	Protocol	Destination	Source	Time	
Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN] 5555 → 59127	56	TCP	127.0.0.1	127.0.0.1	3.273533	60
Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN, ACK] 59127 → 5555	56	TCP	127.0.0.1	127.0.0.1	3.273579	61
Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.273609	62
Seq=1 Ack=1 Win=65280 Len=4 [PSH, ACK] 5555 → 59127	48	TCP	127.0.0.1	127.0.0.1	3.273670	63
Seq=1 Ack=5 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.273683	64
Seq=1 Ack=5 Win=65280 Len=8 [PSH, ACK] 59127 → 5555	52	TCP	127.0.0.1	127.0.0.1	3.273982	65
Seq=5 Ack=9 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.273994	66
Seq=5 Ack=9 Win=65280 Len=4 [PSH, ACK] 5555 → 59127	48	TCP	127.0.0.1	127.0.0.1	3.274029	67
Seq=9 Ack=9 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.274046	68
Seq=9 Ack=9 Win=65280 Len=22 [PSH, ACK] 5555 → 59127	66	TCP	127.0.0.1	127.0.0.1	3.274065	69
Seq=9 Ack=31 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.274072	70
Seq=9 Ack=31 Win=65280 Len=14 [PSH, ACK] 59127 → 5555	58	TCP	127.0.0.1	127.0.0.1	3.274097	71
Seq=31 Ack=23 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.274105	72
Seq=31 Ack=23 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.274274	73
Seq=23 Ack=55 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.274289	74
Seq=55 Ack=23 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.274367	75
Seq=23 Ack=79 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.274379	76
Seq=79 Ack=23 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.274456	77
Seq=23 Ack=103 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.274466	78
Seq=103 Ack=23 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.274613	79
Seq=23 Ack=127 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.274623	80
Seq=23 Ack=127 Win=65280 Len=14 [PSH, ACK] 59127 → 5555	58	TCP	127.0.0.1	127.0.0.1	3.274747	81
Seq=127 Ack=37 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.274757	82
Seq=37 Ack=127 Win=65280 Len=14 [PSH, ACK] 59127 → 5555	58	TCP	127.0.0.1	127.0.0.1	3.274840	83
Seq=127 Ack=51 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.274849	84
Seq=51 Ack=127 Win=65280 Len=14 [PSH, ACK] 59127 → 5555	58	TCP	127.0.0.1	127.0.0.1	3.274990	85
Seq=127 Ack=65 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275002	86
Seq=127 Ack=65 Win=65280 Len=22 [PSH, ACK] 5555 → 59127	66	TCP	127.0.0.1	127.0.0.1	3.275129	87
Seq=65 Ack=149 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275142	88
Seq=65 Ack=149 Win=65280 Len=14 [PSH, ACK] 59127 → 5555	58	TCP	127.0.0.1	127.0.0.1	3.275159	89
Seq=149 Ack=79 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275171	90
Seq=149 Ack=79 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.275236	91
Seq=79 Ack=173 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275244	92
Seq=173 Ack=79 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.275319	93
Seq=79 Ack=197 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275327	94
Seq=79 Ack=197 Win=65280 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.275386	95
Seq=197 Ack=92 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275393	96
Seq=197 Ack=92 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.275423	97
Seq=92 Ack=221 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275436	98
Seq=221 Ack=92 Win=65280 Len=24 [PSH, ACK] 5555 → 59127	68	TCP	127.0.0.1	127.0.0.1	3.275505	99
Seq=92 Ack=245 Win=65280 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275514	100
Seq=92 Ack=245 Win=65280 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.275623	101
Seq=245 Ack=105 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275630	102
Seq=245 Ack=105 Win=65280 Len=25 [PSH, ACK] 5555 → 59127	69	TCP	127.0.0.1	127.0.0.1	3.275724	103
Seq=105 Ack=270 Win=65024 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.275740	104
Seq=270 Ack=118 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275750	105
Seq=270 Ack=118 Win=65280 Len=25 [PSH, ACK] 5555 → 59127	69	TCP	127.0.0.1	127.0.0.1	3.275823	106
Seq=118 Ack=295 Win=65024 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275834	107
Seq=118 Ack=295 Win=65024 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.275877	108
Seq=295 Ack=131 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275885	109
Seq=295 Ack=131 Win=65280 Len=25 [PSH, ACK] 5555 → 59127	69	TCP	127.0.0.1	127.0.0.1	3.275913	110
Seq=131 Ack=320 Win=65024 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.275921	111
Seq=131 Ack=320 Win=65024 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.275970	112
Seq=320 Ack=144 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.275978	113
Seq=144 Ack=320 Win=65024 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.276043	114
Seq=320 Ack=157 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.276050	115
Seq=320 Ack=157 Win=65280 Len=14 [PSH, ACK] 5555 → 59127	58	TCP	127.0.0.1	127.0.0.1	3.276080	116
Seq=157 Ack=334 Win=65024 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.276088	117
Seq=157 Ack=334 Win=65024 Len=13 [PSH, ACK] 59127 → 5555	57	TCP	127.0.0.1	127.0.0.1	3.276314	118
Seq=334 Ack=170 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.276324	119
Seq=334 Ack=170 Win=65280 Len=4 [PSH, ACK] 5555 → 59127	48	TCP	127.0.0.1	127.0.0.1	3.276684	120
Seq=170 Ack=338 Win=65024 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.276696	121
Seq=170 Ack=338 Win=65024 Len=5 [PSH, ACK] 59127 → 5555	49	TCP	127.0.0.1	127.0.0.1	3.276737	122
Seq=338 Ack=175 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.276745	123
Seq=338 Ack=175 Win=65280 Len=0 [FIN, ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.276922	124
Seq=175 Ack=339 Win=65024 Len=0 [ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.276932	125
Seq=175 Ack=339 Win=65024 Len=0 [FIN, ACK] 59127 → 5555	44	TCP	127.0.0.1	127.0.0.1	3.277253	126
Seq=339 Ack=176 Win=65280 Len=0 [ACK] 5555 → 59127	44	TCP	127.0.0.1	127.0.0.1	3.277274	127

נבצע Follow TCP Stream:

```
SIN

SIN/ACK

ACK
GetMaxMsgSize:20,True

MaxMsgSize:20

M0:Hello, This is a tes
M1:t message for Assign
M2:ment 3. We are testi
M3:ng reliable transpor

ACK:-1:MAX:21
ACK:-1:MAX:22
ACK:-1:MAX:23

GetMaxMsgSize:20,True

MaxMsgSize:20

M0:Hello, This is a tes
M1:t message for Assign

ACK:3:MAX:21

M2:ment 3. We are testi
M3:ng reliable transpor

ACK:3:MAX:22

M4:t with sliding window

ACK:3:MAX:23

M5:, ACKs, retransmissio

ACK:4:MAX:24

M6:ns and dynamic messag

ACK:5:MAX:25
ACK:6:MAX:26

M7:e size.

ACK:7:MAX:27

FIN

ACK:
```

ניתן לראות שמוצגת תעבורת TCP מלאה של חיבור אחד, הכוללת 31 חבילות בסך הכול: 17 חבילות שנשלחו מהלקוח, 14 חבילות שנשלחו מהשרת ו-21 חילופי הודעות (Turns) ברמת האפליקציה. ניתן לראות את כל שלבי הפרוטוקול: לחיצת יד (Handshake), בקשת גודל הודעה מקסימלי, ושליחת הודעה מפוצלת למקטעים באמצעות חלון הזזה (Sliding Window). הייחודיות בריצה זו היא העדכון הדינמי של גודל המקטע: השרת מגדיל את גודל ההודעה המותר תוך כדי תנועה (מ-20 ועד 27 בתים). בתגובה, הלקוח מבצע אריזה מחדש של הנתונים שטרם אושרו, וממשיך את השידור עם מספור מעודכן, עד לסיום החיבור באמצעות הודעת FIN.

מצב נוסף שנרצה להראות, הוא כאשר dynamic_message_size = false:

נשנה ב config.txt את ערך הdynamic message size כך:

dynamic message size: false

צד השרת:

```
[server] connected to ('127.0.0.1', 55912)
[server] Simulating packet loss
[server] Dropped line: b'SIN'
[server] Simulating packet loss
[server] Dropped line: b'ACK'
[server] sent MaxMsgSize 20
[server] received segment 0 (20 bytes)
[server] received segment 1 (20 bytes)
[server] received segment 2 (20 bytes)
[server] received segment 3 (20 bytes)
[server] Simulating packet loss
[server] Dropped line: b'M4:t with sliding windo'
[server] received segment 5 (20 bytes)
[server] received segment 6 (20 bytes)
[server] received segment 7 (13 bytes)
[server] Simulating packet loss
[server] Dropped line: b'M4:t with sliding windo'
[server] received segment 5 (20 bytes)
[server] received segment 6 (20 bytes)
[server] received segment 7 (13 bytes)
[server] Simulating packet loss
[server] Dropped line: b'M4:t with sliding windo'
[server] received segment 5 (20 bytes)
[server] received segment 6 (20 bytes)
[server] received segment 7 (13 bytes)
[server] received segment 4 (20 bytes)
[server] received segment 5 (20 bytes)
[server] received segment 6 (20 bytes)
[server] received segment 7 (13 bytes)
[server] client closed connection, stopping ACKs
```

ניתן לראות שהשרת הוגדר בתצורה סטטית (dynamic_message_size: False), עם גודל הודעה קבוע של 20 בתים. השרת דימה איבוד חבילות ו"איבד" את חבילה 4 שלוש פעמים. לאחר שהלקוח שלח את החבילות שוב הוא קיבל 8 מקטעים (Segments 0-7). כל המקטעים הגיעו בגודל המקסימלי (20 בתים), למעט המקטע האחרון (13 בתים) שהכיל את שארית ההודעה. לא היו שינויים בגודל ההודעה במהלך הריצה, וההודעה הורכבה מחדש והודפסה בהצלחה ללא צורך באריזה מחדש או בטיפול בחריגות גודל.

צד הלקוח:

```
C:\dev\assignment3\Assignment_3>py client.py --config C:\dev\assignment3\Assignment_3\config.txt
[Client] Initial max message size = 20 bytes
[Client] Sent segment 0
[Client] Sent segment 1
[Client] Sent segment 2
[Client] Sent segment 3
[Client] Received ACK 0
[Client] Sent segment 4
[Client] Received ACK 1
[Client] Received ACK 2
[Client] Received ACK 3
[Client] Sent segment 5
[Client] Sent segment 6
[Client] Sent segment 7
[Client] Received duplicate ACK 3 (count=1)
[Client] Received duplicate ACK 3 (count=2)
[Client] Received duplicate ACK 3 (count=3)
[Client] Triple duplicate ACKs for 3 -> fast retransmit
[Client] Sent segment 4
[Client] Sent segment 5
[Client] Sent segment 6
[Client] Sent segment 7
[Client] Received duplicate ACK 3 (count=1)
[Client] Received duplicate ACK 3 (count=2)
[Client] Received duplicate ACK 3 (count=3)
[Client] Triple duplicate ACKs for 3 -> fast retransmit
[Client] Sent segment 4
[Client] Sent segment 5
[Client] Sent segment 6
[Client] Sent segment 7
[Client] Received duplicate ACK 3 (count=1)
[Client] Received duplicate ACK 3 (count=2)
[Client] Received duplicate ACK 3 (count=3)
[Client] Triple duplicate ACKs for 3 -> fast retransmit
[Client] Sent segment 4
[Client] Sent segment 5
[Client] Sent segment 6
[Client] Sent segment 7
[Client] Received ACK 7
[Client] FIN acknowledged by server
[Client] Connection closed
[Client] Transmission complete
```

הלקוח התחבר לשרת והחל לשלוח את המידע בחלון הזזה (Sliding Window) בגודל 4. תחילה נשלח חלון מלא (0, 1, 2, 3). לאחר קבלת ה-ACK הראשון (ACK 0), הלקוח "החליק" את החלון ושלח את המקטע הבא (4), וכן הלאה (ACK 1 ואז שליחת 5). מכיוון שהתקבל 3ACK משולש הלקוח שולח שוב את מקטע 4. מכיוון שהשרת איבד את החבילה עוד פעמיים הלקוח נאלץ לשלוח את מקטע 4 עוד פעמיים. בדוף מתקבל ACK7 כלומר כל המקטעים הגיעו ולכן הלקוח סוגר את החיבור. אין שינויים בגודל ה-Max Message Size (שנשאר 20 לאורך כל הדרך).

בוצעה הקלטה עבור המקרה הנ"ל(קובץ pcap_static_mode.pcap בזיפ):
נבצע סינון לפי tcp.port == 5555:

Info	ength	Protocol	Destination	Source	Time	
Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN] 5555 → 55912	56	TCP	127.0.0.1	127.0.0.1	2.770838	46
Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM [SYN, ACK] 55912 → 5555	56	TCP	127.0.0.1	127.0.0.1	2.770879	47
Seq=1 Ack=1 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	2.770907	48
Seq=1 Ack=1 Win=65280 Len=4 [PSH, ACK] 5555 → 55912	48	TCP	127.0.0.1	127.0.0.1	2.770967	49
Seq=1 Ack=5 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	2.770978	50
Seq=5 Ack=1 Win=65280 Len=4 [PSH, ACK] 5555 → 55912	48	TCP	127.0.0.1	127.0.0.1	5.780601	87
Seq=1 Ack=9 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.780625	88
Seq=9 Ack=1 Win=65280 Len=4 [PSH, ACK] 5555 → 55912	48	TCP	127.0.0.1	127.0.0.1	5.780637	89
Seq=1 Ack=13 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.780643	90
Seq=1 Ack=13 Win=65280 Len=8 [PSH, ACK] 55912 → 5555	52	TCP	127.0.0.1	127.0.0.1	5.780677	91
Seq=13 Ack=9 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.780683	92
Seq=9 Ack=13 Win=65280 Len=8 [PSH, ACK] 55912 → 5555	52	TCP	127.0.0.1	127.0.0.1	5.780694	93
Seq=13 Ack=17 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.780698	94
Seq=13 Ack=17 Win=65280 Len=4 [PSH, ACK] 5555 → 55912	48	TCP	127.0.0.1	127.0.0.1	5.780724	95
Seq=17 Ack=17 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.780729	96
Seq=17 Ack=17 Win=65280 Len=23 [PSH, ACK] 5555 → 55912	67	TCP	127.0.0.1	127.0.0.1	5.780745	97
Seq=17 Ack=40 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.780750	98
Seq=17 Ack=40 Win=65280 Len=14 [PSH, ACK] 55912 → 5555	58	TCP	127.0.0.1	127.0.0.1	5.781462	99
Seq=40 Ack=31 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.781473	100
Seq=40 Ack=31 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.781819	101
Seq=31 Ack=64 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.781829	102
Seq=64 Ack=31 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.781952	103
Seq=31 Ack=88 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.781959	104
Seq=88 Ack=31 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.782055	105
Seq=31 Ack=112 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.782061	106
Seq=112 Ack=31 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.782152	107
Seq=31 Ack=136 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.782158	108
Seq=31 Ack=136 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.782716	109
Seq=136 Ack=37 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.782731	110
Seq=37 Ack=136 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.782941	111
Seq=136 Ack=43 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.782950	112
Seq=43 Ack=136 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.783070	113
Seq=136 Ack=49 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.783077	114
Seq=49 Ack=136 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.783185	115
Seq=136 Ack=55 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.783192	116
Seq=136 Ack=55 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.783343	117
Seq=55 Ack=160 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.783353	118
Seq=160 Ack=55 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.783825	119
Seq=55 Ack=184 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.783834	120
Seq=184 Ack=55 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.783934	121
Seq=55 Ack=208 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.783941	122
Seq=208 Ack=55 Win=65280 Len=17 [PSH, ACK] 5555 → 55912	61	TCP	127.0.0.1	127.0.0.1	5.784887	123
Seq=55 Ack=225 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.784899	124
Seq=55 Ack=225 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.785385	125
Seq=225 Ack=61 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.785398	126
Seq=61 Ack=225 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.785590	127
Seq=225 Ack=67 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.785599	128
Seq=67 Ack=225 Win=65280 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.787313	129
Seq=225 Ack=73 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.787326	130
Seq=225 Ack=73 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.788858	131
Seq=73 Ack=249 Win=65280 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.788872	132
Seq=249 Ack=73 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.789011	133
Seq=73 Ack=273 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.789020	134
Seq=273 Ack=73 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.789127	135
Seq=73 Ack=297 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.789134	136
Seq=297 Ack=73 Win=65280 Len=17 [PSH, ACK] 5555 → 55912	61	TCP	127.0.0.1	127.0.0.1	5.789426	137
Seq=73 Ack=314 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.789436	138
Seq=73 Ack=314 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.789923	139
Seq=314 Ack=79 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.789935	140
Seq=79 Ack=314 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.790058	141
Seq=314 Ack=85 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.790066	142
Seq=85 Ack=314 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.790164	143
Seq=314 Ack=91 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.790171	144
Seq=314 Ack=91 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.790675	145
Seq=91 Ack=338 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.790686	146
Seq=338 Ack=91 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.790810	147
Seq=91 Ack=362 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.790817	148
Seq=362 Ack=91 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.790914	149
Seq=91 Ack=386 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.790921	150
Seq=386 Ack=91 Win=65280 Len=17 [PSH, ACK] 5555 → 55912	61	TCP	127.0.0.1	127.0.0.1	5.791012	151
Seq=91 Ack=403 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.791019	152
Seq=91 Ack=403 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.791353	153
Seq=403 Ack=97 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.791363	154
Seq=97 Ack=403 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.791469	155
Seq=403 Ack=103 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.791475	156
Seq=103 Ack=403 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.791564	157
Seq=403 Ack=109 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.791570	158
Seq=403 Ack=109 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.794660	159
Seq=109 Ack=427 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.794678	160
Seq=427 Ack=109 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.796095	161
Seq=109 Ack=451 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.796117	162
Seq=451 Ack=109 Win=65280 Len=24 [PSH, ACK] 5555 → 55912	68	TCP	127.0.0.1	127.0.0.1	5.796329	163
Seq=109 Ack=451 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.796339	164
Seq=475 Ack=115 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.796356	165
Seq=115 Ack=475 Win=65024 Len=0 [ACK] 55912 → 5555	44	TCP	127.0.0.1	127.0.0.1	5.796361	166
Seq=475 Ack=115 Win=65280 Len=17 [PSH, ACK] 5555 → 55912	61	TCP	127.0.0.1	127.0.0.1	5.796445	167
Seq=115 Ack=492 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.796566	168
Seq=492 Ack=121 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.796577	169
Seq=492 Ack=121 Win=65280 Len=4 [PSH, ACK] 5555 → 55912	48	TCP	127.0.0.1	127.0.0.1	5.796735	170
Seq=121 Ack=496 Win=65024 Len=6 [PSH, ACK] 55912 → 5555	50	TCP	127.0.0.1	127.0.0.1	5.796795	171
Seq=496 Ack=127 Win=65280 Len=0 [ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.796809	172
Seq=496 Ack=127 Win=0 Len=0 [RST, ACK] 5555 → 55912	44	TCP	127.0.0.1	127.0.0.1	5.796886	173

נבצע Follow TCP Stream:

```
SIN
SIN
SIN

SIN/ACK
SIN/ACK

ACK
GetMaxMsgSize:20,False

MaxMsgSize:20

M0:Hello, This is a tes
M1:t message for Assign
M2:ment 3. We are testi
M3:ng reliable transpor

ACK:0
ACK:1
ACK:2
ACK:3

M4:t with sliding windo
M5:w, ACKs, retransmiss
M6:ions and dynamic mes
M7:sage size.

ACK:3
ACK:3
ACK:3

M4:t with sliding windo
M5:w, ACKs, retransmiss
M6:ions and dynamic mes
M7:sage size.

ACK:3
ACK:3
ACK:3
```

```
M4:t with sliding windo
M5:w, ACKs, retransmiss
M6:ions and dynamic mes
M7:sage size.

ACK:3
ACK:3
ACK:3

M4:t with sliding windo
M5:w, ACKs, retransmiss
M6:ions and dynamic mes

ACK:7

M7:sage size.

ACK:7

FIN

ACK:7
```

הפעם התעבורה כוללת 45 הודעות לוגיות בסך הכול: 26 הודעות שנשלחו מהלקוח ן-19 הודעות שנשלחו מהשרת. ניתן לראות את כל שלבי הפרוטוקול: לחיצת יד (Handshake), בקשת גודל הודעה מקסימלי (שנקבע על 20 בתים), שליחת הודעה מפוצלת למקטעים באמצעות חלון הזזה (Sliding Window) ושליחה מחודשת של חלון כאשר חבילה אבדה. הייחודיות בריצה זו היא הסטטיות של גודל המקטע: בניגוד לדוגמה הקודמת, כאן גודל ההודעה נשמר קבוע (20 בתים) לאורך כל הריצה. הודעות האישור (ACKs) אינן מכילות עדכוני גודל, והשידור מתבצע בצורה ליניארית וחלקה (מקטעים M0 עד M7) ללא אריזה מחדש, עד לסיום החיבור באמצעות הודעת FIN.